

Design Pattern

...

2025 J5

Object Pool, lancement TinyRPG

Rappel des épisodes précédents :



Développement sur le jeu TANKS ! Avec des modifications de ce jeu

Nous avons parlé de :

- C# avancé, SOLID, Architecture, Approche ECS
- Coroutine / AsyncAwait, LINQ, ScriptableObject
- Design patterns :
 - GameLoop, Singleton, ServiceLocator, Proxy
 - FactoryMethod, Prototype, Commande
 - Flyweight, StateMachine, DirtyFlag etc.
 - <https://gameprogrammingpatterns.com/contents.html>

Un dernier pour la route

Object Pool



Object Pool

Intérêt :

- Nos bullet, sont instanciés et détruites constamment
- Beaucoup d'allocation et de "libérations" mémoire
- Fragmentation de la mémoire -> Garbage Collector qui s'affole

Le principe

On connaît l'instance que l'on doit précharger, dans notre cas Bullet.

On en crée plusieurs (ni trop, ni pas assez)

Dès que l'on a besoin d'un bullet -> Au lieu de l'instancier, on demande à la pool de nous en fournir un.

La bullet est réinitialisée à un état initial et fourni à l'appelant.

Une fois la vie de bullet terminée, au lieu de la destroy, elle retourne dans la pool en attendant qu'on ai à nouveau besoin d'elle.

(exemple “DemoObjectPool.scene”)

Création d'un object pool à vous !

On va tweak une arme pour tirer bien trop rapidement.

Puis ajouter un object pool pour gérer la mémoire au mieux

30 mins

Des design patterns il pourrait en avoir de nouveaux

Si on trouve qu'une pratique remplit parfaitement les besoins il est tout à fait possible de le voir formaliser. Voire qu'il atterrisse dans le langage si vraiment ça vaut le coup

Alterable, un concept pas si mal

Dans un jeu, les statistiques de nos personnages changent souvent.

Level up, ATK + 10

Une nouvelle arme, ATK + 20

Un sort de magie temporaire, ATK +10%

Le soucis c'est l'ordre de cette application : ATK + 10 puis ATK + 20 puis ATK +10% donnera un résultat différent de : ATK+10, ATK+10%, ATK+20.

Problèmes de nos stats

Egalement, lorsque l'on retire une amélioration on pourrait avoir un soucis.

Level up, ATK + 10

Une nouvelle arme, ATK + 20

Un sort de magie temporaire, ATK +10%

J'ai ATK+10, ATK+20, ATK+10% ... Mais je désequipe mon arme ... Le buff fini par s'estomper ... Difficile de revenir en arrière, le buff prenait en compte le +20 de l'arme mais en enlevant le buff, on doit prendre en compte ce changement.

Une solution : conserver toutes les modifications

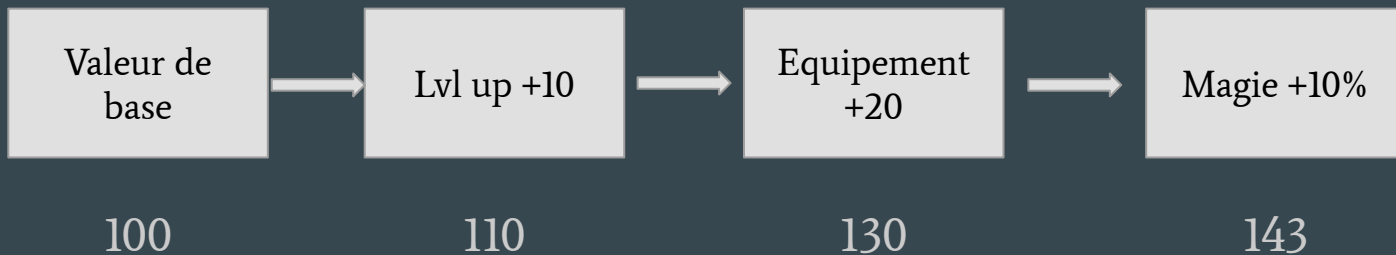
Chaque transformation de la stat sera une fonction qui prend une valeur d'entrée, et retourne sa valeur modifiée.

`WeaponImprovement(int valeurDeBase) => valeurDeBase + 10;`

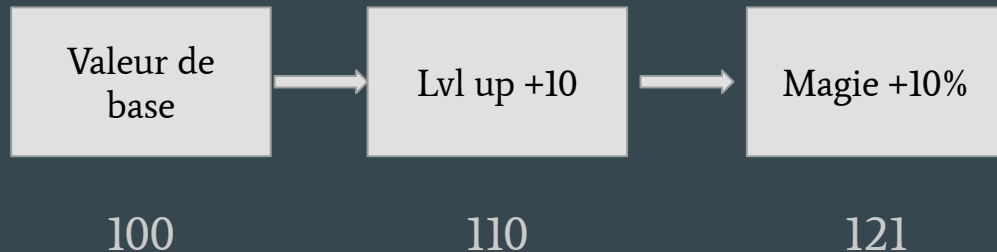
`MagicalBuff(int valeurDeBase) => valeurDeBase * 1.1f;`

Alterable

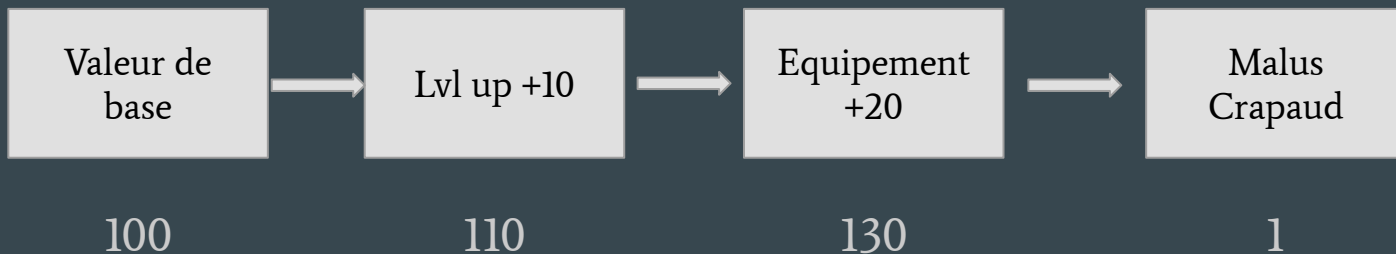
Exemple 1



Exemple 2



Alterable, on pourrait même forcer une valeur en bout de chaîne



On obtient une liste de “Transformateur”

Quand on veut récupérer la valeur de notre stat, on traverse toutes les fonctions pour obtenir la valeur finale.

Avantage : si on doit supprimer une des transformations ça revient juste à supprimer un élément d'une liste, on ne passera plus par se “transformateur” pour obtenir la valeur finale.

Inconvénient : Empreinte mémoire un peu plus importante et évaluation de toutes les fonctions (possible d'ajouter un cache + dirty flag mais il faut le prévoir)

Exemple live

Test sur une stat arbitraire de notre tank

Maintenant ... à vous de prendre votre envol.

Nous allons travailler sur un nouveau projet : TinyRPG

Projet créé par mes soins, mais ... bien vide.

Vous allez devoir implémenter de nouvelles features en respectant tout ce dont on a parlé ensemble, propreté du code, organisation, SOLID etc.

Et surtout employer les design patterns de votre choix pour créer vos mécaniques.

Exemple :

Je veux tester la création du pattern de la commande :

Je rajoute dans le jeu la mécanique de retour dans le temps !

Je veux tester la state machine : Création d'IA de nos ennemis

Singleton ? Injecteur de dépendance ? ça peut être le contexte de tester Zenject ou VContainer (attention c'est beaucoup plus complexe d'un coup)

AsyncAwait ? Profitons de ce bac à sable.

LINQ ? Essayons avec un leaderboard ou une selection automatique de la target lors d'une attaque ?

Modalité :

Un projet en duo

L'idée est d'intégrer un maximum de design pattern et surtout de pouvoir les argumenter, on doit pouvoir expliquer pourquoi on a organisé nos scripts de cette manière.

En début de chaque matinée on pourra débriefer et revoir ce que vous voulez pour mieux appréhender le cours.

Rendu prévu : ce vendredi 12h, via la réponse à un formulaire fourni en dernière journée.

Rendu :

Le rendu sera donc votre projet sur un repo github

ET un document listant tout ce que vous avez produit comme design pattern.

Expliquez clairement vos choix de design, les features ajoutés et pattern employés pour résoudre tout ceci.

Vous pouvez modifier ce que vous voulez du jeu, si vous préférez transformer le projet en shooter pour exploiter une object pool n'hésitez pas. On pourrait préférer un ennemis qui simule un sorte de bullet hell pour ce design pattern aussi, vous choisissez.

Notation :

La note se fera en fonction des design patterns mis en place, certains sont bien plus balaise à mettre en place. Une valorisation sera faite sur les designs/techniques les plus complexe.

Le document sera également important puisqu'il me permet de comprendre ce que vous avez intégré des design patterns et de leur utilisation.

Globalement dès que votre travail est régulier, bien expliqué et organisé vous aurez dans les 12-16 / 20.

Les notes de 16 à 20 seront justifié lors d'utilisation de design pattern plus complexe (et bien exécuté)

Design Pattern “standard”

Proxy

Archi basé sur les ScriptableObject (SOAP)

Object Pool

State Machine

Injection par champs / méthode

LINQ

“Alterable”

Design Pattern “complexe”

Commande

Injection de dépendance (VContainer)

BehaviorTree

A vous

Si vous avez des question n'hésitez pas bien sûr.

Et profitez de ce projet pour expérimenter, c'est un bac à sable pour tester ce que vous aurez pas forcément fait sur un projet en prod.